MYSQL AVANCÉ POUR INGÉNIEURS

Du Scolaire au Professionnel de Haut Niveau

- Master DATA & Développement

École INGETIS Paris

INTRODUCTION

Bienvenue dans ce cours intensif qui va vous transformer d'étudiants maîtrisant les bases de MySQL en ingénieurs capables de concevoir, optimiser et maintenir des systèmes de bases de données à l'échelle professionnelle.

Pourquoi ce cours est différent : Vous maîtrisez déjà le MCD, le MLD, les formes normales et les requêtes JOIN. C'est excellent, mais c'est le strict minimum pour survivre en production. Ce qui fait la différence entre un junior et un senior, c'est la capacité à :

- Écrire du SQL qui « scale » à des millions de lignes
- Comprendre ce qui se passe « sous le capot » de MySQL
- Architecturer des systèmes hybrides SQL/NoSQL
- Intégrer les bases de données dans des pipelines DevOps modernes

Structure du cours : Ce cours est organisé en 3 axes stratégiques représentant 7 heures de formation intensive :

AXE	DURÉE	OBJECTIFS CLÉS
Performance & Under the Hood	3 heures	Indexation avancée, tuning MySQL, partitionnement
Architecture & Modernité	2 heures	Merise vs DDD, hybridation SQL/NoSQL, données temporelles
DevOps & Automatisation	2 heures	Versioning BD, sécurité avancée, cas pratique Crash Test

AXE 1 : PERFORMANCE & UNDER THE HOOD

Durée : 3 heures

Objectif : Passer de SQL qui « marche » à SQL qui « scale ». En Master, on n'écrit plus du code pour faire plaisir au prof, on écrit du code qui doit tenir face à des millions de lignes et des milliers de requêtes par seconde.

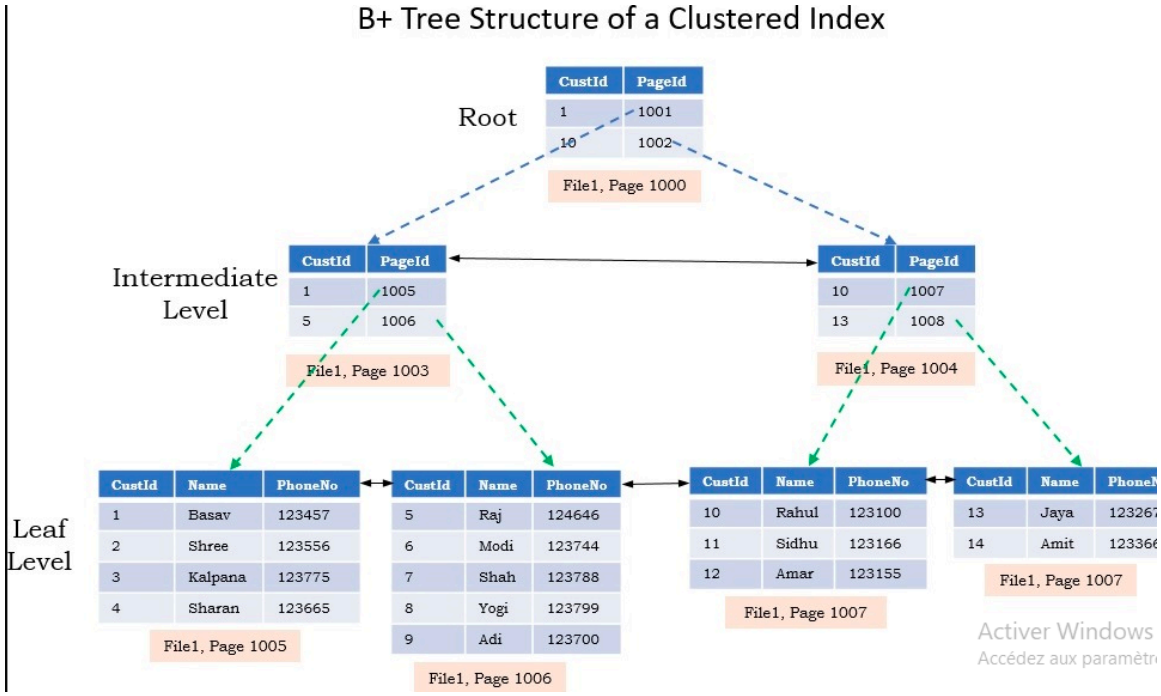
1.1 L'Indexation Avancée : Au-delà du « ça accélère »

Le mythe à casser : « Les index, c'est magique, ça rend tout plus rapide ». **FAUX.** Un index mal placé peut ralentir vos INSERT et UPDATE de 30 à 50 %. Parlons de ce qui se passe réellement sous le capot.

1.1.1 Les structures d'index : B-Tree vs Hash

B-Tree (par défaut dans InnoDB) :

- Structure : Arbre équilibré qui maintient les données triées
- Idéal pour : Les recherches de plages (BETWEEN, >, <), les ORDER BY, les recherches exactes
- Coût : O(log n) pour la recherche, mais occupe plus d'espace
- Mise à jour : Relativement coûteuse car nécessite de maintenir l'équilibre de l'arbre



Hash Index (disponible dans MEMORY/HEAP tables) :

- Structure : Table de hachage directe
- Idéal pour : Recherches d'égalité exacte uniquement (WHERE id = 5)
- Coût : O(1) pour les recherches exactes
- Limitation : Ne fonctionne PAS pour les recherches de plages, LIKE, ORDER BY

⚠️ **PIÈGE CLASSIQUE :** Créer un index sur une colonne avec très peu de valeurs distinctes (ex: sexe, statut actif/inactif) peut être **contre-productif**. L'optimiseur MySQL préférera souvent un full table scan.

1.1.2 EXPLAIN ANALYZE : Votre meilleur ami

EXPLAIN vous montre *ce que MySQL prévoit de faire*. **EXPLAIN ANALYZE** vous montre *ce qu'il a réellement fait* avec les coûts en temps réel.

Exemple pratique :

```
EXPLAIN ANALYZE
SELECT * FROM commandes
WHERE date_commande BETWEEN '2024-01-01' AND '2024-12-31';
```

Que chercher dans le résultat :

- type: ALL → Full table scan = CATASTROPHE sur une grande table
- type: index → Scan d'index complet = Mieux, mais pas optimal
- type: range → Utilise un index pour une plage = BON
- type: ref → Utilise un index non-unique = BON
- type: const → Recherche par clé primaire/unique = EXCELLENT
- rows: Nombre de lignes examinées. Si c'est proche du total de la table, problème !
- Extra: Using filesort → Tri en mémoire/disque = coûteux
- Extra: Using temporary → Création de table temporaire = très coûteux

💡 **EXERCICE PRATIQUE :** Vous allez créer une table de 1 million de lignes, lancer une requête sans index, observer le temps d'exécution, puis créer l'index approprié et mesurer la différence. Attendez-vous à des gains de 100x à 1000x.

1.2 Le Tuning MySQL : Le Buffer Pool d'InnoDB

Le **Buffer Pool** est le cœur de MySQL. C'est une zone mémoire où InnoDB met en cache les données et les index. Si vos données tiennent en mémoire, MySQL vole. Sinon, il rampe.

1.2.1 Dimensionnement du Buffer Pool

Règle d'or : Sur un serveur dédié à MySQL, allouez 70-80 % de la RAM au Buffer Pool. Sur un serveur partagé, soyez plus conservateur (50-60 %).

Configuration (dans my.cnf) :

```
[mysqld]
innodb_buffer_pool_size = 4G
innodb_buffer_pool_instances = 4
```

⚠️ **Points critiques :**

- La taille doit être un multiple de innodb_buffer_pool_chunk_size × innodb_buffer_pool_instances
- Par défaut : chunk_size = 128M, donc avec 4 instances, la taille doit être un multiple de 512M
- MySQL ajuste automatiquement à la hausse si nécessaire, mais soyez prévenus !

1.2.2 Le mécanisme LRU (Least Recently Used)

InnoDB utilise un algorithme LRU amélioré pour gérer le cache. Le Buffer Pool est divisé en deux sous-listes :

- New sublist (jeunes pages) : ~5/8 du buffer, contient les pages fréquemment accédées
- Old sublist (vieilles pages) : ~3/8 du buffer, contient les pages moins utilisées

Pourquoi cette complexité ? Pour éviter qu'un full table scan (ex: mysqldump) n'éjecte toutes vos données chaudes du cache. Les nouvelles pages entrent d'abord dans la old sublist et ne migrent vers la new sublist que si elles sont réutilisées.

1.2.3 SSD vs HDD : Impact sur la configuration

SSD (Solid State Drive) :

- Temps d'accès : ~0.1 ms (100x plus rapide qu'un HDD)
- IOPS : 10,000+ opérations/seconde

- Configuration : Augmentez innodb_io_capacity à 2000-5000

HDD (Hard Disk Drive) :

- Temps d'accès : ~10 ms
- IOPS : ~100-200 opérations/seconde
- Configuration : Gardez innodb_io_capacity à 200

🎯 **EN PRODUCTION :** Un SSD correctement configuré peut gérer 10 à 50 fois plus de charge qu'un HDD. Si vous devez choisir entre doubler la RAM ou passer de HDD à SSD, prenez le SSD.

1.3 Le Partitionnement : Gérer les tables massives

Vous avez 50 millions de lignes ? 100 millions ? Bienvenue dans le monde réel. À ce niveau, une seule table devient ingérable. Le partitionnement permet de diviser logiquement une table en plusieurs morceaux physiques.

1.3.1 Partitionnement Horizontal (par lignes)

Principe : Diviser une table en plusieurs partitions selon une clé (date, région, ID).

Exemple : Partitionnement par date

```
CREATE TABLE commandes (
  id INT,
  date_commande DATE,
  montant DECIMAL(10,2),
  PRIMARY KEY (id, date_commande)
)
PARTITION BY RANGE (YEAR(date_commande)) (
  PARTITION p2022 VALUES LESS THAN (2023),
  PARTITION p2023 VALUES LESS THAN (2024),
  PARTITION p2024 VALUES LESS THAN (2025),
  PARTITION p_future VALUES LESS THAN MAXVALUE
);
```

Avantages :

- Pruning : MySQL ignore automatiquement les partitions inutiles
- Maintenance : Vous pouvez archiver ou supprimer une partition entière en une seule opération
- Performance : Chaque partition a ses propres index

1.3.2 Sharding (Partitionnement applicatif)

Quand même le partitionnement ne suffit plus, on passe au **sharding** : répartir les données sur *plusieurs serveurs MySQL indépendants*.

Exemple : Application e-commerce avec 500 millions de clients :

- Shard 1 : Clients dont l'ID se termine par 0-3
- Shard 2 : Clients dont l'ID se termine par 4-6
- Shard 3 : Clients dont l'ID se termine par 7-9

⚠️ **COMPROMIS :** Le sharding se gère au niveau applicatif. Vous ne pouvez plus faire de JOIN entre shards. C'est puissant mais contraignant.

AXE 2 : ARCHITECTURE & MODERNITÉ

Durée : 2 heures

Objectif : Merise n'est pas mort, il a juste évolué. Découvrez comment les concepts de modélisation relationnelle se marient avec les architectures modernes (microservices, NoSQL, systèmes distribués).

2.1 Merise vs Domain-Driven Design (DDD)

Merise est perçu comme « vieux » par certains, mais ses principes sont toujours valables. Le DDD (Domain-Driven Design) apporte une vision complémentaire axée sur le métier.

2.1.1 Le MCD et les Aggregate Roots

Merise (MCD) : Focus sur les entités et leurs relations (cardinalités 1:N, N:M, etc.)

DDD (Aggregate Root) : Focus sur les *frontières transactionnelles*. Un Aggregate Root est une entité qui garantit la cohérence d'un ensemble d'objets liés.

Exemple : Système de commande e-commerce

- MCD : Entités Commande (1) ↔ (N) LigneCommande, avec clés étrangères
- DDD : Commande est l'Aggregate Root, LigneCommande n'existe que via Commande. On ne peut pas modifier une LigneCommande sans passer par sa Commande parente.

💡 **PONT ENTRE LES DEUX :** Le MCD vous aide à identifier les entités et leurs relations. Le DDD vous aide à décider comment découper votre base de données en microservices. Chaque Aggregate Root peut devenir un microservice avec sa propre base.

2.2 L'Hybridation SQL/NoSQL : Le meilleur des deux mondes

MySQL gère nativement le type JSON depuis la version 5.7.8 (2015). En 2025, c'est devenu un standard dans l'industrie. Au lieu de choisir entre MySQL et MongoDB, vous pouvez avoir les deux dans une même base.

2.2.1 Le type JSON dans MySQL

Pourquoi JSON ?

- Données semi-structurées : Métadonnées, configurations, logs
- Flexibilité : Pas besoin de migration de schéma à chaque nouveau champ
- Performance : MySQL stocke JSON en format binaire optimisé

Exemple : Système de produits e-commerce

```
CREATE TABLE produits (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nom VARCHAR(255) NOT NULL,
  prix DECIMAL(10,2) NOT NULL,
  categorie VARCHAR(100),
  -- Partie stricte : données critiques

  attributs JSON,
  -- Partie flexible : métadonnées variables
  -- Ex: {"couleur": "rouge", "taille": "L", "poids": 1.5}

  INDEX idx_categorie (categorie),
  INDEX idx_prix (prix)
);
```

Requêter le JSON :

```
-- Extraire une valeur
SELECT nom, attributs->>'$.couleur' AS couleur
FROM produits
WHERE attributs->>'$.taille' = 'L';

-- Créer un index sur un champ JSON
CREATE INDEX idx_couleur ON produits((attributs->>'$.couleur'));
```

2.2.2 Quand utiliser JSON vs Colonnes relationnelles ?

Critère	Colonnes relationnelles	Colonnes JSON
Validation stricte	✓ Excellente	✗ Faible
Performance des requêtes	✓ Excellente	Moyenne (avec index)
Flexibilité du schéma	✗ Faible	✓ Excellente
Coût de migration	✗ Élevé	✓ Très faible

🎯 **RÈGLE DE DÉCISION :** Si c'est critique pour le business (prix, stock, identité), utilisez des colonnes relationnelles. Si c'est des métadonnées qui évoluent souvent (tags, préférences, config), utilisez JSON.

2.3 La Gestion des Données Temporelles : L'Historisation

Problème réel : Un client change d'adresse. Vous mettez à jour votre table clients. Deux ans plus tard, il veut réimprimer une facture. L'adresse de livraison sur la facture doit être celle qu'il avait *à l'époque*, pas celle d'aujourd'hui. Comment gérer ça ?

2.3.1 SCD Type 2 (Slowly Changing Dimensions)

Solution : Créer une nouvelle ligne à chaque changement, en conservant l'historique complet.

Schéma :

```
CREATE TABLE clients_historise (
  id INT AUTO_INCREMENT PRIMARY KEY,
  client_id INT NOT NULL,
  nom VARCHAR(255),
  adresse VARCHAR(500),
  ville VARCHAR(100),
  code_postal VARCHAR(10),

  date_debut DATE NOT NULL,
  date_fin DATE DEFAULT NULL,
  est_actuel BOOLEAN DEFAULT TRUE,

  INDEX idx_client_actuel (client_id, est_actuel),
  INDEX idx_dates (date_debut, date_fin)
);
```

Logique :

- Chaque ligne représente un état du client à un moment donné
- date_debut : Date à laquelle cet état est devenu valide
- date_fin : Date à laquelle cet état a cessé d'être valide (NULL si toujours actuel)
- est_actuel : Booléen pour accélérer les recherches de l'état actuel

Requête pour obtenir l'adresse à une date donnée :

```
-- Adresse du client 123 au moment de sa commande du 2023-05-15
SELECT adresse, ville, code_postal
FROM clients_historise
WHERE client_id = 123
  AND date_debut <= '2023-05-15'
  AND (date_fin IS NULL OR date_fin > '2023-05-15');
```

💡 **CAS D'USAGE :** E-commerce (adresses de livraison historiques), RH (salaires et promotions), assurances (polices et garanties), comptabilité (historique des prix).

AXE 3 : DEVOPS & AUTOMATISATION

Durée : 2 heures

Objectif : Un ingénieur ne tape pas son SQL dans PHPMyAdmin. Les bases de données doivent être versionnées, automatisées et sécurisées comme n'importe quel autre code.

3.1 Le Versioning de Base de Données : Flyway et Liquibase

Le problème : Votre application évolue. Vous ajoutez une colonne en dev, vous la testez en staging, puis vous la déployez en prod. Comment garantir que tous vos environnements restent synchronisés ? Comment annuler un changement qui pose problème ?

3.1.1 Flyway : Simplicité et convention

Philosophie : « Convention over configuration ». Vous écrivez des scripts SQL avec un nommage précis, Flyway s'occupe du reste.

Convention de nommage :

- V : Migration versionnée (ex: V001__Create_users_table.sql)
- U : Migration de rollback (ex: U001__Undo_users_table.sql)
- R : Migration répétable (ex: vues, procédures stockées)

Exemple de migration :

```
-- V001__Create_users_table.sql
CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Commandes Flyway :

```
flyway migrate    # Applique toutes les migrations en attente
flyway info       # Affiche l'état des migrations
flyway validate   # Vérifie la cohérence
flyway clean      # ATTENTION: Supprime tout (dev only!)
```

Avantages :

- Simple : Vous écrivez du SQL pur
- Versionné : Chaque migration est numérotée et traçable

- Déterministe : Les migrations s'appliquent dans l'ordre, une seule fois

3.1.2 Liquibase : Flexibilité et portabilité

Philosophie : « Database-agnostic ». Vous définissez vos changements en XML/YAML/JSON, Liquibase génère le SQL adapté à votre SGBD (MySQL, PostgreSQL, Oracle, etc.).

Exemple de changelog (XML) :

```
<databaseChangeLog>
  <changeSet id="1" author="karim">
    <createTable tableName="users">
      <column name="id" type="int" autoIncrement="true">
        <constraints primaryKey="true"/>
      </column>
      <column name="email" type="varchar(255)">
        <constraints nullable="false" unique="true"/>
      </column>
      <column name="created_at" type="timestamp"
              defaultValueComputed="CURRENT_TIMESTAMP"/>
    </createTable>
  </changeSet>
</databaseChangeLog>
```

Avantages :

- Portabilité : Un seul fichier fonctionne sur MySQL, PostgreSQL, Oracle
- Rollback intégré : Liquibase peut générer automatiquement les rollbacks
- Diff : Comparer deux bases et générer les migrations manquantes

3.1.3 Flyway vs Liquibase : Lequel choisir ?

Critère	Flyway	Liquibase
Simplicité	✓ Très simple	Courbe d'apprentissage
Multi-SGBD	Non (SQL spécifique)	✓ Oui
Rollback	Manuel (Team edition)	✓ Automatique
Diff/Sync	Non (Desktop edition)	✓ Oui

🎯 **RECOMMANDATION :** Pour un projet MySQL unique, Flyway est plus simple et direct. Pour un projet multi-SGBD ou avec des besoins de rollback complexes, Liquibase est plus adapté.

3.2 Sécurité Avancée : Au-delà des injections SQL

Les injections SQL, c'est la base. En Mastere, on attend de vous que vous maîtrisiez des concepts de sécurité plus profonds : chiffrement, audit, gestion des privilèges.

3.2.1 Encryption at Rest

Problème : Quelqu'un vole votre disque dur. Les fichiers InnoDB sont en clair. Vos données sensibles sont compromises.

Solution : MySQL 5.7+ supporte le chiffrement transparent des tablespaces InnoDB.

Activation (dans my.cnf) :

```
[mysqld]
early-plugin-load=keyring_file.so
keyring_file_data=/var/lib/mysql-keyring/keyring

# Chiffrer par défaut toutes les nouvelles tables
default_table_encryption=ON
```

Chiffrer une table existante :

```
ALTER TABLE users ENCRYPTION='Y';
```

3.2.2 Gestion des Privilèges : Principe du moindre privilège

❌ **ERREUR CLASSIQUE :** Donner GRANT ALL à l'utilisateur applicatif.

✓ **BONNE PRATIQUE :** Créer des utilisateurs avec des privilèges minimaux.

Exemple :

```
-- Utilisateur pour l'application web (lecture/écriture)
CREATE USER 'app_user'@'app_server_ip' IDENTIFIED BY 'strong_password';
GRANT SELECT, INSERT, UPDATE ON ecommerce.* TO 'app_user'@'app_server_ip';

-- Utilisateur pour les backups (lecture seule)
CREATE USER 'backup_user'@'localhost' IDENTIFIED BY 'another_strong_pass';
GRANT SELECT, LOCK TABLES, RELOAD ON *.* TO 'backup_user'@'localhost';

-- Utilisateur pour l'analytics (lecture seule)
CREATE USER 'analytics_user'@'%' IDENTIFIED BY 'yet_another_pass';
GRANT SELECT ON ecommerce.commandes TO 'analytics_user'@'%';
GRANT SELECT ON ecommerce.produits TO 'analytics_user'@'%';
```

3.3 Cas Pratique : Le « Crash Test »

Scénario : Vous héritez d'une base de données « propre » sur le papier (MCD nickel, 3ème forme normale respectée), mais qui ne tient pas en production. Votre mission : diagnostiquer et corriger.

Problème concret :

- Table produits avec 10 millions de lignes
- Table categories avec 50 lignes
- Table marques avec 200 lignes

- Table fournisseurs avec 500 lignes

Requête typique de l'application (affichage d'un produit) :

```
SELECT p.nom, p.prix, c.nom AS categorie, m.nom AS marque, f.nom AS fournisseur
FROM produits p
JOIN categories c ON p.categorie_id = c.id
JOIN marques m ON p.marque_id = m.id
JOIN fournisseurs f ON p.fournisseur_id = f.id
WHERE p.id = 12345;
```

Symptômes :

- Temps de réponse : 200-500 ms par requête
- Sous forte charge (1000 requêtes/sec), le serveur sature

Votre diagnostic (avec EXPLAIN ANALYZE) :

- Les 4 JOIN sont coûteux même si individuellement chaque table est petite
- Pour afficher UN produit, MySQL charge 4 tables supplémentaires
- Les données de categories, marques et fournisseurs changent rarement

Solution : Dénormalisation contrôlée

1. Ajoutez des colonnes dénormalisées :

```
ALTER TABLE produits
ADD COLUMN categorie_nom VARCHAR(100),
ADD COLUMN marque_nom VARCHAR(100),
ADD COLUMN fournisseur_nom VARCHAR(100);
```

2. Remplissez-les :

```
UPDATE produits p
JOIN categories c ON p.categorie_id = c.id
JOIN marques m ON p.marque_id = m.id
JOIN fournisseurs f ON p.fournisseur_id = f.id
SET p.categorie_nom = c.nom,
    p.marque_nom = m.nom,
    p.fournisseur_nom = f.nom;
```

3. Modifiez la requête :

```
SELECT nom, prix, categorie_nom, marque_nom, fournisseur_nom
FROM produits
WHERE id = 12345;
```

Résultat :

- Temps de réponse : 5-10 ms (gain de 20x à 100x)
- Plus de JOIN en lecture

- Serveur peut tenir 10,000+ requêtes/sec

⚠️ **COMPROMIS :** Vous devez maintenant synchroniser les noms dénormalisés quand une catégorie/marque/fournisseur change. Mais comme ça arrive rarement, c'est un compromis acceptable.

💡 **LEÇON :** La normalisation parfaite n'est pas toujours la bonne solution en production. Un bon ingénieur sait quand et comment dénormaliser intelligemment pour gagner en performance.

CONCLUSION

Ce que vous devez retenir :

- Performance : Comprendre comment MySQL fonctionne sous le capot (B-Tree, Buffer Pool, LRU) fait la différence entre un junior et un senior
- Architecture : Merise reste valable, mais l'hybridation SQL/NoSQL et la gestion des données temporelles sont indispensables en 2025
- DevOps : Versionner votre base de données avec Flyway/Liquibase n'est plus une option, c'est une nécessité
- Pragmatisme : La théorie c'est bien, mais la production vous forcera à faire des compromis. Savoir quand dénormaliser ou quand partitionner est un art

🎯 **CHALLENGE FINAL :** Appliquez tout ce que vous avez appris sur un projet réel. Créez une base de données pour un système de réservation de salles (100,000+ réservations par mois), optimisez-la, versionnez-la avec Flyway, chiffrez les données sensibles, et préparez-vous à la défendre devant un jury. C'est ça, être un ingénieur.

Bon courage à tous !

— Karim, École INGETIS Paris